# AI Project Report: AI-Driven Food Distribution System

Matteo Bravin Giacomo Prioli Martino Bissiato

June 8, 2026

## 1 Project Overview and Objectives

The proposed system is an intelligent allocation engine designed to optimize the distribution of food resources from a central warehouse to various beneficiaries (e.g., hospitals, canteens, family homes, families). The primary objective is to maximize human satisfaction by meeting precise caloric and nutritional requirements while respecting strict warehouse inventory limits and entity urgency constraints.

To achieve this, the system uses a hybrid approach: a **Genetic Algorithm (GA)** for global combinatorial optimization, coupled with **Heuristic Prioritization** and a **Memetic Post-Processing (Greedy Clean-up)** phase to ensure zero-waste and strict constraint adherence.

## 2 Core Design Choices

### 2.1 Why Supabase?

The choice of Supabase as the Backend-as-a-Service (BaaS) provides several strategic advantages:

- **Relational Integrity:** Unlike NoSQL databases, Supabase is built on PostgreSQL. This allows strict relational mapping between `Beneficiaries`, `People`, and `Products`, ensuring data consistency (e.g., via foreign keys).
- **Cloud-Native & Python Friendly:** The `supabase-py` client allows seamless fetching of initial state data (the "world state") directly into Python dataclasses without building custom API routing.
- **Real-time Potential:** Although currently running as a batch script, Supabase's real-time capabilities leave the door open for future real-time inventory updates.

### 2.2 Why a Genetic Algorithm (GA)?

Food distribution is a classic **Multi-Objective Knapsack/Resource Allocation Problem**, which is NP-hard.

- **Massive Search Space:** Assigning hundreds of discrete product units to dozens of beneficiaries creates a combinatorial explosion that deterministic algorithms (like Linear Programming) struggle to solve quickly when non-linear constraints (like balanced macros) are introduced.
- **Multi-Objective Nature:** The GA allows for a highly customized fitness function that balances conflicting goals: maximizing caloric satisfaction, penalizing excessive sugars/salts, ensuring sufficient proteins, and prioritizing urgent entities.

### 2.3 Why a Memetic / Greedy Clean-up Phase?

Standard GAs can sometimes converge on near-optimal solutions that leave minor warehouse remnants. The system implements a **Memetic** approach via `apply_greedy_cleanup`. This deterministic algorithm takes the GA's output and assigns remaining inventory to entities based on priority, with a hard constraint: no entity can exceed 100% of its caloric requirement. This ensures zero waste of useful food while preventing overfeeding.

## 3 Detailed Code Architecture

The codebase is highly modular, separating data access, domain logic, algorithmic processing, and analytics.

### 3.1 Data Models (`beneficiaries.py`, `people.py`, `products.py`)

These modules define Python `@dataclass` structures that mirror the Supabase schema.

- `Person`: Contains biometrics (age, gender), a foreign key (`beneficiary_id`), and a dynamically calculated `priority` attribute.
- `Beneficiary`: Represents the entity. Categorized by `type` (e.g., Mensa, Ospedale).
- `Product`: Contains macronutrient data and the available `quantity` in the warehouse.

### 3.2 Prioritization Engine (`priorities.py`)

Before the GA runs, the system calculates how "urgent" a person and an entity are. For an individual person, the priority is calculated using weighted normalized scores:

$$P_{person} = (w_{age} \cdot S_{age}) + (w_{entity} \cdot S_{entity}) + (w_{cal} \cdot S_{cal}) + (w_{gender} \cdot S_{gender}) \quad (1)$$

Where the weights are strictly defined: $w_{age} = 0.40$, $w_{entity} = 0.30$, $w_{cal} = 0.15$, $w_{gender} = 0.15$.

The priority of a Beneficiary is then aggregated based on the people associated with it:

$$P_{entity} = \left( \frac{\sum P_{person}}{N} \cdot 1.0 \right) + (N \cdot 0.2) \quad (2)$$

Where $N$ is the number of people in the entity. This ensures that larger entities, and entities with highly vulnerable individuals, receive higher priority.

## 3.3 The Genetic Algorithm (`genetic_algorithm.py` & `fitness.py`)

- **Chromosome Representation:** A nested dictionary where `chromosome[beneficiary_id][pro` equals the allocated quantity.
- **Initialization:** Generates a random population, assigning products to random entities.
- **Fitness Function (`evaluate_fitness`):** The algorithm starts with an arbitrarily high score (e.g., 1000000.0) and applies penalties:
  1. **Stock Violations:** Exceeding warehouse capacity is heavily penalized ($-5000$ per unit).
  2. **Caloric Gap:** Deviations from the target caloric requirement are penalized, scaled by the entity's priority:
  $$\text{Penalty}_{cal} = |C_{alloc} - C_{target}| \cdot 0.5 \cdot P_{entity} \tag{3}$$
  3. **Nutritional Constraints:** Hard penalties for sugar $> 50g$ per person, salt $> 6g$, or proteins $< 60g$.
- **Softmax Repair (`repair_chromosome`):** To resolve stock limit violations, the algorithm removes excess products from entities. To decide *who* loses a product, it uses a Softmax probability distribution:
  $$W = e^{-P_{entity} \cdot \left(\frac{\max(0, C_{target} - C_{alloc})}{C_{target}}\right)} \tag{4}$$
  This brilliantly ensures that entities with high priority and high unmet needs have a mathematically lower probability ($W$) of having food taken away during the repair phase.

## 3.4 Post-Processing & Exports (`exports.py`)

After optimization, the system provides transparent reporting:

- **Analytics Generation:** Uses `matplotlib` to plot convergence curves, entity allocation vs. requirements, and personal satisfaction rates.
- **Data Export:** Generates structured `CSV` and `JSON` files containing detailed routing plans. This makes the AI's output interoperable with external logistics systems.

## 3.5 Orchestration (`main.py`)

This is the entry point. It sequentially:

1. Connects to Supabase.
2. Calculates heuristic priorities.
3. Runs the Genetic Algorithm.
4. Applies the Greedy Clean-up.
5. Prints a terminal summary and triggers the exports.

# 4 Conclusion

The system successfully bridges modern cloud infrastructure (Supabase) with advanced heuristic AI (Genetic + Memetic algorithms). By encoding human vulnerability and nutritional requirements into a strict mathematical fitness space, it provides an ethical, automated, and mathematically sound distribution strategy.